

TechSara — Salesforce Query Accuracy Fix

Why counts come back wrong (200 / 224 instead of 225), the root cause, and the exact production-safe fix — including full-list retrieval and XML/CSV export without hitting Salesforce governor limits.

In one line

The data is correct and the model is correct. Counts are wrong only because a count is sometimes taken from a **capped list of rows** (200 on the live path) instead of a real **SELECT COUNT()**, and because a query occasionally runs against **live Salesforce** instead of the local warehouse. No model training is required.

Evidence (reproduced on the current build)

What was asked	Answer	Correct?
DuckDB warehouse — direct query	225	Correct
AI — count only	225	Correct
AI — list and count together	200	Wrong
AI — count (RecordType join variant)	224	Wrong

The true number is **225**. Both 200 and 224 are bugs, from two independent causes explained below.

1. Root Cause

1a. Why 200 — the count was read off a capped list

When a list is requested together with the count, the request is sometimes served from **live Salesforce** rather than the DuckDB warehouse. The live path enforces a hard row ceiling, and the count is then derived from the number of rows returned.

```
File: orchestrator/app/core/salesforce.py — line 30
MAX_ROWS = 200 # hard ceiling on rows returned
...
text = f"{text} LIMIT {MAX_ROWS}" # a row query is capped at 200
rows = [_clean(r) for r in records][:MAX_ROWS] # result re-sliced to 200
```

The reported total is literally `len(rows)`. If rows are capped at 200, the count is 200 — even though 225 exist. This is why **count-only** works (it runs on the warehouse with a 100,000 cap and a real COUNT wrap) but **list+count** breaks.

1b. Why 224 — an unnecessary RecordType inner join dropped one row

The count query the model wrote in that case was:

```
SELECT COUNT(*) FROM Interview__c i
JOIN RecordType rt ON i.RecordTypeId = rt.Id
WHERE rt.Name = 'Interview' -- drops rows
AND i.Interview_Status__c = 'Completed'
AND i.Round__c = 'Final'
```

The **INNER** join with `rt.Name = 'Interview'` silently removes any Interview row whose record type is null or not literally named 'Interview'. The interviews pack documents an `Is_Loop_Round__c` flag — loop interviews (several rounds in one record) are exactly the kind of row a record-type filter drops. Result: 225 → 224. The correct hand-written query used **LEFT JOIN** and no record-type filter, and returned 225.

1c. Why it went live at all — the warehouse was momentarily unavailable

The warehouse holds the full data with no 200 cap. The engine falls back to live Salesforce only when DuckDB is **busy** (a sync write every ~5 minutes briefly locks it) or a table/field name does not resolve. So the list+count run happened to land during a warehouse-busy window and was pushed onto the 200-capped live path.

The two independent bugs, precisely

Bug A (200): a count taken from a row list that the live path caps at 200. **Bug B (224):** a RecordType inner join that removes one row. Bug A is a *counting-mechanism* problem; Bug B is a *query-shape* problem. They must both be fixed — neither is a model-quality issue.

2. The Fix — five changes

Ordered smallest-and-safest first. Changes 1–2 are low-risk and can ship immediately; 3–5 are the durable, general fix for every object.

Change 1 — Brain rule (fixes the 224; zero risk, hot-reloads)

File: `brain/packs/kh-external-interviews.vaml` — inside the `rules:` block

- For counts, ALWAYS use `SELECT COUNT(*)`. Never count by listing rows.
- Do NOT join or filter `Interview__c` by `RecordType` for status/round questions. Status and Round already identify the rows (a `RecordType` inner join dropped one row: 225 became 224). Use `LEFT JOIN` if needed and never filter on it.
- For a list AND a count together, run TWO queries with the SAME WHERE; report the count from `COUNT(*)`, the list from the row query.

Then run the ship gate: `python orchestrator/scripts/validate_packs.py`. No restart needed — packs hot-reload.

Change 2 — Raise the on-screen preview cap

File: `orchestrator/app/core/exports.py` — line 20

```
PREVIEW_ROW_CAP = 500 # before
PREVIEW_ROW_CAP = 2000 # after (your 225 now shows in full)
```

Keep it bounded (2,000–5,000). Do **not** set 100000 — a browser cannot paint that many rows at once. Requires an orchestrator restart.

Change 3 — Add governor-safe pagination (the core)

File: `orchestrator/app/core/salesforce.py`. Add two bounds next to `MAX_ROWS`, then a new function after `query_more()`. It reuses the existing already-guarded `run_soql_page()` and `query_more()`.

```
# next to MAX_ROWS = 200
LIST_HARD_CAP = 50_000 # absolute safety ceiling for one list
LIST_PAGE_SANITY = 500 # max queryMore round-trips

async def run_soql_all(soql: str, hard_cap: int = LIST_HARD_CAP) -> dict:
    """Run a guarded SELECT and follow queryMore until the whole
    result set is fetched (or hard_cap is reached)."""
    first = await run_soql_page(soql) # existing, guarded
    rows = list(first["rows"])
    next_url = first.get("next_records_url") or ""
    truncated = False
    pages = 0
    while next_url:
        if len(rows) >= hard_cap or pages >= LIST_PAGE_SANITY:
            truncated = True
            break
        page = await query_more(next_url) # existing, guarded
        rows.extend(page["rows"])
        next_url = page.get("next_records_url") or ""
        pages += 1
    return {
        "soql": first["soql"],
        "rows": rows[:hard_cap],
        "total_size": first.get("total_size") or len(rows),
        "done": not next_url,
        "truncated": truncated,
    }
```

Change 4 — Route list requests through the paginated fetch

File: orchestrator/app/engines/live_sf.py — the two run_soql() call sites (lines 187 and 205)

```
import re # at top of file if not already present

# Aggregates (COUNT/SUM/...): one call, true total, no pagination.
# Row lists: page through everything up to the hard cap.
is_aggregate = bool(re.match(
    r"\s*SELECT\s+(COUNT|SUM|AVG|MIN|MAX)\s*\(", soql, re.I))
if is_aggregate:
    page = await salesforce.run_soql_page(soql)
    return page["soql"], page["rows"]
result = await salesforce.run_soql_all(soql)
return result["soql"], result["rows"]
```

Now a live list returns the full set (all 225), and a live count returns the true total via COUNT().

Change 5 — XML export of the full list

File: orchestrator/app/core/exports.py. Generated server-side from the full row set, so the 200 screen/live cap does not apply (it uses the 100,000 export cap).

```
import xml.sax.saxutils as _xml

def rows_to_xml(rows: list[dict], root="records",
    row_tag="record") -> str:
    capped, was_capped = apply_export_cap(rows) # existing 100k helper
    out = ['<?xml version="1.0" encoding="UTF-8"?>']
    out.append(f'<{root}>{" truncated=1" if was_capped else ""}>')
    for r in capped:
        out.append(f" <{row_tag}>")
        for k, v in r.items():
            tag = _xml.escape(str(k))
            val = "" if v is None else _xml.escape(str(v))
            out.append(f" <{tag}>{val}</{tag}>")
        out.append(f" </{row_tag}>")
    out.append(f"</{root}>")
    return "\n".join(out)
```

Wire it wherever CSV download is offered, passing the full paged rows (from run_soql_all), not the 200-capped display rows.

3. Salesforce Governor Limits — why this is safe

The most important point for a production org: fetching the full list this way does **not** threaten Salesforce limits.

The key fact about queryMore

queryMore (following nextRecordsUrl) is **not a new query**. Salesforce counts only the **initial** query against the daily API/SOQL limit; the follow-up pages continue the **same cursor**. Paging 225 rows is ~1 logical query, not 225. Even 50,000 rows is a handful of REST round-trips on one query.

Concern	Safe?	Why
Daily API / SOQL call limit	Yes	queryMore continues ONE query; a full list $\approx$ 1 query, not N
2,000-row SOQL batch limit	Yes	pagination follows nextRecordsUrl through every batch
COUNT of any size	Yes	aggregate, no LIMIT, one small call
Runaway pull (millions of rows)	Yes	LIST_HARD_CAP 50k + page sanity 500 stop it
Write safety on production	Yes	read-only integration user; guard blocks non-SELECT
Browser freeze on a huge list	Yes	screen shows a bounded window; full data goes to export

Prefer the warehouse for very large history

For genuinely large historical pulls, the DuckDB warehouse has **no API limit at all** and is far faster. Use live Salesforce only when the data must be fresher than the last ~5-minute sync. Even then, counts use COUNT() and lists paginate — both governor-safe.

4. What Improves After the Fix

- **Counts are always correct** for every object — they come from `SELECT COUNT()`, never from a capped row list.
- **Lists return the full set** (e.g. all 225) on the live path, not 200, via governor-safe pagination.
- **On-screen display** shows the full list for ordinary sizes (up to 2,000) instead of stopping at 500.
- **XML and CSV export** carry the complete result (up to 100,000 rows), independent of the screen/live cap.
- **The 224 disappears** — the model no longer adds a `RecordType` inner join that drops rows.
- **Salesforce stays safe** — the design actually reduces live API pressure by preferring the warehouse and using `COUNT()` for totals.
- **No training, no model change** — the model already emits the correct fields and values.

Rollout order

Step	Change	File	Risk
1	Brain rule (fixes 224) + <code>validate_packs.py</code>	<code>kb-external-interviews.yaml</code>	None
2	Preview cap 500 → 2000	<code>core/exports.py</code>	Low
3	Add <code>run_soql_all</code> + bounds	<code>core/salesforce.py</code>	Medium
4	Route lists to <code>run_soql_all</code>	<code>engines/live_sf.py</code>	Medium
5	Add <code>rows_to_xml</code> + wire download	<code>core/exports.py</code>	Low

Acceptance test

In Salesforce mode, ask:

```
give me interview count where interview status is completed
and round is final, also give me the full list
```

Expected after Changes 1–4: **count = 225** (from `COUNT`), **list = 225 rows** (paginated, not 200), both consistent, and no governor-limit impact. With Change 5, the XML/CSV export contains all 225 rows.

Bottom line

The system is well-built and the model is accurate. The wrong numbers came from a counting mechanism (`len(rows)` on a 200-capped live result) and one query-shape mistake (a `RecordType` inner join) — not from misunderstanding the question. Compute counts with `COUNT()`, paginate lists with `queryMore` (one logical query, governor-safe), bound the screen and export, and forbid the unnecessary join. That fully closes the issue for every object.